

Aula 13: Large Language Models (LLMs)

De Arquiteturas Generativas a Fine-Tuning Eficiente (LoRA)

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

Novembro de 2025

Roteiro da Aula

- 1 Estudo de Caso: “Chatbot” com Seq2Seq e Atenção
- 2 A Arquitetura GPT
- 3 Pré-treinamento Generativo vs. Masking
- 4 A Era dos LLMs e o Fine-Tuning
- 5 Encerramento

Estudo de Caso: Arquitetura Chatbot Q&A I

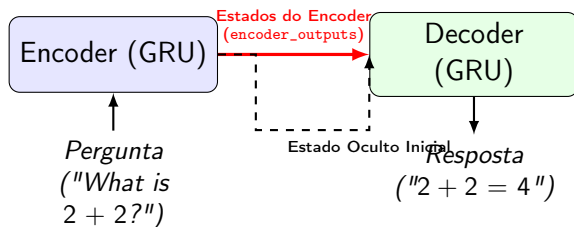
Objetivo: Implementar um Modelo Seq2Seq com Atenção

Para aplicar os conceitos de atenção, vamos construir um modelo que aprende a mapear Perguntas para Respostas. Usaremos a arquitetura Encoder-Decoder com uma RNN (GRU) e um mecanismo de atenção.

Componentes Chave:

- **Encoder (GRU):** Lê a pergunta e gera um estado oculto para cada palavra.
- **Decoder (GRU):** Gera a resposta, uma palavra de cada vez.
- **Atenção:** A cada passo, o Decoder "consulta" os estados do Encoder para focar nas palavras relevantes da pergunta.

Estudo de Caso: Arquitetura Chatbot Q&A II



Estudo de Caso: Componente 1 - O Encoder I

O Encoder (EncoderRNN)

A sua única função é ler a sequência de entrada (pergunta) e produzir duas coisas: os estados ocultos de *cada* passo, e o estado oculto *final*.

```
1 class EncoderRNN(nn.Module):
2     def __init__(self, input_size, hidden_size, embed_size):
3         super(EncoderRNN, self).__init__()
4         self.embedding = nn.Embedding(input_size, embed_size, ..
5         self.gru = nn.GRU(embed_size, hidden_size,
6                             batch_first=True, num_layers=2)
7
8     def forward(self, input_seq):
9         embedded = self.embedding(input_seq)
10        outputs, hidden = self.gru(embedded)
```

Estudo de Caso: Componente 1 - O Encoder II

```
11         # outputs: [batch, seq_len, hidden_size]
12         # hidden: [num_layers, batch, hidden_size]
13     return outputs, hidden
```

Analogia QKV

No nosso modelo RNN-Attention, o **Encoder** prepara a "memória" para a atenção:

- `outputs` (todos os estados ocultos) vão servir como as **Keys** (chaves) e os **Values** (valores). Eles representam a informação disponível na entrada.
- `hidden` (o último estado) vai inicializar o Decoder.

Estudo de Caso: Componente 2 - A Lógica da Atenção I

A Consulta (Query) do Decoder

A cada passo, o Decoder pergunta: "No que devo focar-me agora?". Esta pergunta é a sua **Query**. A atenção usa esta Query para calcular um **vetor de contexto**.

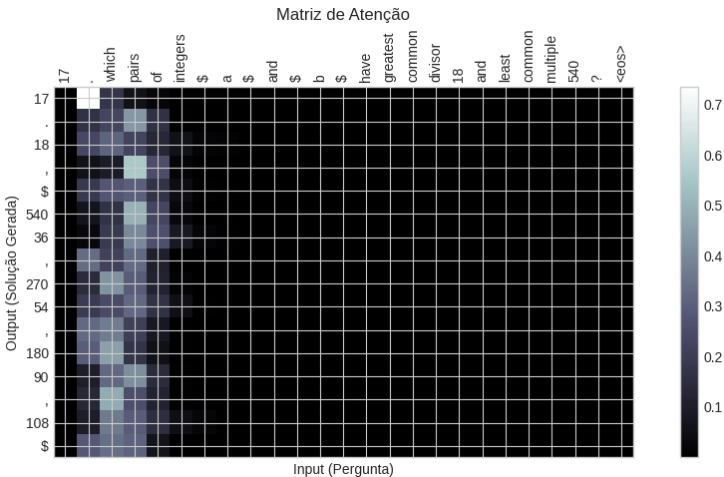
```

1  # Isto é o que acontece dentro do módulo Attention
2  # a cada passo do decoder:
3
4  # 1. Query (Q) = Estado oculto atual do Decoder
5  #   (Ex: O que estou a escrever agora?)
6  query = decoder_hidden[-1] # [B, H]
7
8  # 2. Keys (K) = Todos os outputs do Encoder
9  #   (Ex: Que palavras existem na pergunta?)
10 keys = encoder_outputs      # [B, S, H]
11
12 # 3. Values (V) = Todos os outputs do Encoder
13 #   (Ex: Qual o significado dessas palavras?)
14 values = encoder_outputs    # [B, S, H]
15
16 # 4. Calcular Scores: (Q * K^T)
17 scores = torch.bmm(query.unsqueeze(1),
18                      keys.transpose(1, 2)) # [B, 1, S]
```

Estudo de Caso: Componente 2 - A Lógica da Atenção II

```
19
20 # 5. Obter Pesos: softmax(scores / sqrt(d_k))
21 weights = F.softmax(scores / (keys.size(-1)**0.5),
22                      dim=-1) # [B, 1, S]
23
24 # 6. Obter Contexto: weights * V
25 context = torch.bmm(weights, values) # [B, 1, H]
```


Estudo de Caso: Componente 2 - A Lógica da Atenção III



Attention

Estudo de Caso: Componente 3 - O Decoder I

O Decoder (AttnDecoderRNN)

O Decoder gera a resposta, um token de cada vez. A cada passo, ele usa o vetor de **contexto** (calculado pela Atenção) para informar a sua próxima decisão.

```

1  class AttnDecoderRNN(nn.Module):
2      def __init__(self, ...):
3          ...
4          self.attention = Attention(hidden_size)
5          # A entrada da GRU é maior: Embedding + Contexto
6          self.gru = nn.GRU(embed_size + hidden_size, ...)
7          self.out = nn.Linear(hidden_size, output_size)
8
9      def forward(self, decoder_input, decoder_hidden, encoder_outputs):
10
11         # 1. Obter embedding do token anterior
12         embedded = self.embedding(decoder_input) # [B, 1, E]
13
14         # 2. CALCULAR O CONTEXTO via Atenção
15         context, attn_weights = self.attention(decoder_hidden,
16                                                encoder_outputs)
17
18         # 3. CONCATENAR embedding e contexto

```

Estudo de Caso: Componente 3 - O Decoder II

```
19 rnn_input = torch.cat((embedded, context), dim=2)
20
21 # 4. Passar o input "informado" pela GRU
22 output, hidden = self.gru(rnn_input, decoder_hidden)
23
24 # 5. Obter logits (previsão da próxima palavra)
25 logits = self.out(output.squeeze(1))
26
27 return logits, hidden, attn_weights
```

Estudo de Caso: O Loop de Treino I

Orquestrando o Modelo (train_epoch)

A função de treino liga tudo, implementando a geração auto-regressiva e o "Teacher Forcing".

```
1  def train_epoch(...):
2      ...
3      for inputs, targets in dataloader:
4          ...
5          # 1. Correr o Encoder UMA VEZ sobre a pergunta
6          encoder_outputs, encoder_hidden = encoder(inputs)
7
8          # 2. Inicializar o Decoder com o estado do Encoder
9          decoder_hidden = encoder_hidden
10         decoder_input = torch.full((...), SOS_TOKEN, ...) # <sos>
11
```

Estudo de Caso: O Loop de Treino II

```
12         batch_loss = 0
13         max_target_len = targets.shape[1]
14
15         # 3. Loop de Geração (Auto-regressivo)
16         for t in range(max_target_len):
17
18             # 4. Correr o Decoder UM PASSO
19             logits, decoder_hidden, _ = decoder(decoder_input,
20                                                  decoder_hidden,
21                                                  encoder_outputs)
22
23             loss = criterion(logits, targets[:, t])
24             batch_loss += loss
25
26             # 5. Teacher Forcing:
27             # Alimentar a RESPOSTA CORRETA como próxima entrada
```

Estudo de Caso: O Loop de Treino III

```
28         decoder_input = targets[:, t].unsqueeze(1)
29         ...
30         batch_loss.backward()
31         ...
```

Ponto de Partida para LLMs

Esta arquitetura Seq2Seq é poderosa, mas **recorrente e sequencial**. A geração é inerentemente lenta (passo a passo). Esta limitação é o que motiva diretamente a arquitetura paralela dos LLMs (Transformers).

Da Tradução para a Geração Pura I

O Transformer original (Vaswani et al., 2017) era um Encoder-Decoder, ótimo para tradução.

Da Tradução para a Geração Pura II

A Ideia do GPT (Generative Pre-Training)

E se quiséssemos apenas um modelo de linguagem? Um modelo que "entende" e "escreve" texto, sem uma entrada e saída separadas?

- **Resposta:** Vamos usar apenas a pilha de **Decoders** do Transformer.
- O Decoder do Transformer é perfeito para geração: ele foi desenhado para prever a próxima palavra de uma sequência.
- A sua "Auto-Atenção" (Self-Attention) é **mascarada**: ao prever a palavra na posição i , ele só pode "olhar" para as palavras nas posições $< i$. Isto evita que o modelo "veja a resposta".

Da Tradução para a Geração Pura III

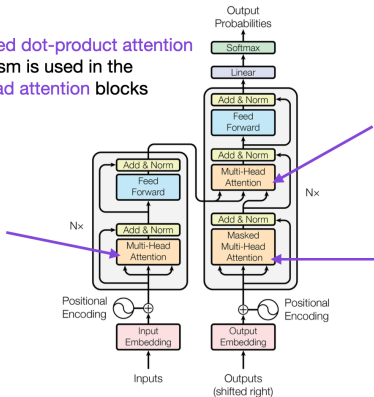
GPT = Pilha de Decoders do Transformer

O BERT usou os Encoders (bidirecionais) para *entender*. O GPT usa os Decoders (auto-regressivos) para *gerar*.

Arquitetura do Decoder do Transformer

Isto é o coração de modelos como o GPT. É uma pilha de blocos idênticos.

The **scaled dot-product attention** mechanism is used in the **multi-head attention** blocks



Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

(A arquitetura GPT é funcionalmente a pilha da direita, sem a pilha da esquerda)

Como Treinar um Modelo Generativo? I

Objetivo do BERT (Encoder)

Masked Language Model (MLM)

- Vê a frase toda (esquerda e direita)
- **Tarefa:** O [MASK] saltou sobre o cão.
- **Ótimo para:** Análise de sentimento, Q&A, classificação.
Tarefas que requerem *entendimento* profundo do contexto completo.

Como Treinar um Modelo Generativo? II

Objetivo do GPT (Decoder)

Causal Language Model (CLM)

- Só pode ver o passado (esquerda)
- **Tarefa:** O gato saltou sobre o ???
- **Ótimo para:** Geração de texto, diálogo, resumo. Tarefas que são *auto-regressivas*.

O Mesmo Objetivo, Duas Abordagens

Ambos são “auto-supervisionados” (não precisam de rótulos humanos), mas o CLM é o que ensina um modelo a **escrever** fluentemente.

Comportamento Emergente: “In-Context Learning” I

Uma das descobertas mais importantes dos LLMs é a aprendizagem “dentro do contexto”.

Comportamento Emergente: “In-Context Learning” II

“Zero-Shot” e “Few-Shot” Learning

O modelo não foi treinado para responder a perguntas. Foi treinado para completar texto. Se o seu prompt (entrada) se parecer com um padrão de Q&A, o modelo irá completá-lo.

Exemplo de Prompt (Few-Shot)

O que é $2+2$? 4.

O que é 5×5 ? 25.

O que é $10-3$?

O modelo reconhece o padrão e prevê "7" como a continuação mais provável. Ele "aprendeu" a tarefa no *prompt*, sem qualquer atualização de pesos.

ICL: A Grande Questão I

Como é que o modelo "aprende" sem gradientes?

O fenómeno de ICL é contra-intuitivo:

- Durante o **treino** (pré-treinamento), o modelo aprende a prever o próximo token. Os seus pesos (W) são atualizados via *gradient descent*.
- Durante a **inferência** (ICL), o modelo "aprende" uma nova tarefa (ex: traduzir, somar) apenas a partir do prompt. Os pesos (W) estão **congelados** e não são atualizados.

ICL: A Grande Questão II

Então, o que está a acontecer?

O ICL não é "treino" no sentido tradicional. É um comportamento emergente. Duas teorias principais tentam explicar isto:

- 1 ICL como **Inferência Bayesiana Implícita**.
- 2 ICL como **Otimização Implícita** (Gradient Descent).

Teoria 1: ICL como Inferência Bayesiana I

O Insight (Xie et al., 2021)

O pré-treinamento não é feito em texto aleatório. Os documentos têm **coerência** e **conceitos latentes** (θ).

- Um documento pode ser sobre "Biografias". O modelo aprende que 'Albert Einstein ->' está associado a 'physicist' e 'German'.
- Outro pode ser "Receitas". O modelo aprende que 'Ingredientes ->' está associado a 'Passos'.

Durante o pré-treinamento, o modelo (LM) aprende a **inferir este conceito latente** θ para prever melhor o próximo token.

Teoria 1: ICL como Inferência Bayesiana II

Como o ICL Acontece

- O "prompt" é tratado como um novo mini-documento.
- O modelo executa **inferência Bayesiana implícita** para descobrir o "conceito" (θ^*) que melhor explica os exemplos no prompt.
- **Exemplo:** Ao ver '(2+2? 4)', '(5x5? 25)', o modelo infere $\theta^* = \text{"tarefa de aritmética"}$.

A previsão final $p(y|\text{prompt})$ é o modelo a prever y dado o conceito θ^* que acabou de inferir:

$$p(y|\text{prompt}) = \int_{\theta} p(y|\text{prompt}, \theta) \underbrace{p(\theta|\text{prompt})}_{\text{Posterior}} d\theta$$

À medida que $p(\theta|\text{prompt})$ colapsa em torno de θ^* , o modelo

Teoria 2: ICL como Otimização Implícita I

O Insight (von Oswald et al., 2022)

Esta é uma visão mecanicista: A passagem *forward* do Transformer é, ela própria, um algoritmo de otimização.

- O Transformer (o "modelo exterior") é treinado com *gradient descent* (SGD) para prever o próximo token.
- No processo, ele **aprende a implementar o seu próprio otimizador** (um "mesa-otimizador") na sua passagem *forward* para resolver a tarefa apresentada no prompt.

Teoria 2: ICL como Otimização Implícita II

A Passagem Forward É Gradient Descent

O modelo trata os exemplos do prompt (x_i, y_i) como um mini-batch de treino para uma *nova* tarefa.

- A tarefa implícita é encontrar um peso W_{task} que minimize $\mathcal{L} = \sum (W_{task} \cdot x_i - y_i)^2$.
- A atualização de W_{task} seria: $W_1 = W_0 - \eta \nabla \mathcal{L}$.
- A previsão para uma nova query x_q é $h = W_1 x_q = (W_0 - \eta \nabla \mathcal{L}) x_q$.

O artigo mostra que a camada de atenção calcula esta atualização $\Delta W = -\eta \nabla \mathcal{L}$ e aplica-a.

ICL como GD: Papel dos Componentes I

ICL como GD: Papel dos Componentes II

Como os Blocos do Transformer Fazem Isto?

Esta teoria dá um papel claro a cada componente do bloco Transformer:

■ Camadas MLP (Feed-Forward):

- Atuam como um "feature extractor" ou "kernel".
- Mapeiam a entrada x para um novo espaço de features $m(x)$ onde a tarefa se torna **linear**.
- Ex: A tarefa não-linear $f(x)$ torna-se uma regressão linear simples $W \cdot m(x)$.

■ Camadas de Self-Attention:

- Executam o **gradient descent** nessa tarefa linear.
- A atenção calcula a atualização ΔW (ou BA no LoRA) necessária para "resolver" a regressão linear $W \cdot m(x)$ usando os exemplos do prompt.

■ Empilhamento (Stacking) de Camadas:

- Cada camada do Transformer aplica **um passo adicional** de

ICL: Síntese das Teorias I

Duas Visões do Mesmo Fenômeno

Ambas as teorias explicam como o ICL emerge do pré-treinamento (previsão do próximo token).

■ Visão Bayesiana (Estatística):

- O ICL é um processo de **inferência**.
- O modelo está a inferir o "conceito latente" (θ^*) mais provável que gerou os exemplos do prompt.
- "Qual é a tarefa (θ) que torna este prompt provável?"

■ Visão Gradient Descent (Mecanicista):

- O ICL é um processo de **otimização implícita**.
- O modelo está a *simular* passos de gradient descent no prompt para "treinar" um modelo linear interno.
- "Como posso calcular o ΔW para resolver esta tarefa?"

ICL: Síntese das Teorias II

Conclusão

O “In-Context Learning” não é mágica. É um comportamento emergente de modelos treinados em dados massivos e estruturados. O Transformer aprende **o algoritmo de como aprender** (seja por inferência ou otimização) diretamente nos seus pesos.

O Problema do Fine-Tuning Total I

Temos um modelo de base (ex: GPT-3 com 175 mil milhões de parâmetros). Queremos que ele seja **muito bom** numa tarefa específica (ex: o nosso Q&A de matemática).

A Abordagem Ingénua: Full Fine-Tuning

- Carregar o modelo de 175B de parâmetros.
- Treiná-lo no nosso dataset de Q&A (como fizemos com a RNN).
- Isto atualizaria **todos os 175 mil milhões de pesos**.

O Problema do Fine-Tuning Total II

Problemas Fatais

- **Custo Proibitivo:** Requer um cluster de GPUs (ex: $8 \times$ A100 80GB). Apenas para *carregar* o modelo ($175B \times 2$ bytes/param = 350GB de VRAM) já é impossível numa única GPU.
- **Armazenamento:** Se tivermos 1000 tarefas, precisamos de 1000 cópias de 350GB do modelo.
- **"Catastrophic Forgetting":** Ao treinar intensamente numa tarefa, o modelo pode "esquecer" o seu conhecimento geral.

Solução: Parameter-Efficient Fine-Tuning (PEFT) I

A Ideia Central

E se pudéssemos adaptar o LLM à nossa tarefa **congelando** todos os seus 175B de parâmetros e treinando apenas um pequeno conjunto de **novos** parâmetros (ex: $< 1\%$ do total)? Isto é o PEFT. A técnica mais popular chama-se **LoRA**.

Solução: Parameter-Efficient Fine-Tuning (PEFT) II

Vantagens do PEFT

- **Custo Baixo:** Treinável numa única GPU (ex: RTX 4090 ou A100).
- **Armazenamento Mínimo:** Para cada tarefa, guardamos apenas os pesos do "adaptador" (ex: 100MB) em vez de um modelo inteiro de 350GB.
- **Sem "Forgetting":** O modelo de base permanece intacto; apenas o "plug-in" da tarefa é ativado.

Como Funciona o LoRA (Low-Rank Adaptation) I

O Insight (Hu et al., 2021)

A hipótese é que a *mudança* nos pesos (ΔW) durante o fine-tuning é de "rank baixo". Isto significa que, embora ΔW seja uma matriz gigante, ela pode ser aproximada por duas matrizes muito mais pequenas (A e B).

Como Funciona o LoRA (Low-Rank Adaptation) II

Formulação Matemática

Seja $W_0 \in \mathbb{R}^{d \times k}$ a matriz de pesos original (congelada). A passagem (forward pass) original é $h = W_0 x$.

O fine-tuning completo (Full Fine-Tuning) aprenderia

$$h = (W_0 + \Delta W)x.$$

O LoRA aproxima ΔW com duas matrizes de "rank baixo", B e A :

$$h = W_0 x + \Delta W x = W_0 x + (BA)x$$

Onde:

- $W_0 \in \mathbb{R}^{d \times k}$ (congelada)
- $A \in \mathbb{R}^{r \times k}$ (treinável)
- $B \in \mathbb{R}^{d \times r}$ (treinável)
- O rank r é um hiperparâmetro minúsculo (ex: 8, 16, 64).

A inicialização de B é feita com zeros, para que $\Delta W = BA = 0$ no

LoRA: O Fluxo do Gradiente I

Como o Treino Funciona

Durante a retropropagação, o gradiente do custo ($\mathcal{L}$) flui apenas para os pesos que importam.

Seja θ o conjunto de todos os parâmetros: $\theta = \{W_0, A, B\}$.

$$\frac{\partial \mathcal{L}}{\partial \theta} = \left\{ \frac{\partial \mathcal{L}}{\partial W_0}, \frac{\partial \mathcal{L}}{\partial A}, \frac{\partial \mathcal{L}}{\partial B} \right\}$$

LoRA: O Fluxo do Gradiente II

Pesos Congelados

Nós **não** calculamos o gradiente para os pesos originais.

$$\frac{\partial \mathcal{L}}{\partial W_0} \equiv 0$$

- Não há atualização.
- Não é preciso guardar os estados do otimizador (ex: Adam) para W_0 .
- Isto poupa a vasta maioria da memória de VRAM.

LoRA: O Fluxo do Gradiente III

Pesos do Adaptador

Nós calculamos o gradiente **apenas** para A e B .

$$\frac{\partial \mathcal{L}}{\partial A} \neq 0 \quad \text{e} \quad \frac{\partial \mathcal{L}}{\partial B} \neq 0$$

- O gradiente flui através de $h = W_0x + (BA)x$.
- Apenas A e B são atualizados pelo otimizador.

LoRA Aplicado à Atenção (Self-Attention) I

Onde aplicar o LoRA?

Não é preciso aplicar A e B a *todas* as camadas lineares. O artigo original descobriu que a maior eficiência vem de adaptar as matrizes de projeção da **Atenção**.

Numa camada de Self-Attention, a entrada X é projetada para Query, Key e Value:

$$Q = XW_Q \quad | \quad K = XW_K \quad | \quad V = XW_V$$

LoRA Aplicado à Atenção (Self-Attention) II

Substituição com LoRA

Os autores aplicam LoRA tipicamente a W_Q e W_V (e por vezes W_K e W_O). A passagem (forward) para a Query torna-se:

$$Q = XW_{0,Q} + X(B_Q A_Q)$$

E para o Value:

$$V = XW_{0,V} + X(B_V A_V)$$

$W_{0,Q}$ e $W_{0,V}$ (os pesos gigantes) são congelados.

A_Q, B_Q, A_V, B_V (as matrizes finas) são treinadas.

O Ecossistema Moderno I

1 Modelo de Base (Foundation Model):

- Um LLM pré-treinado massivo (ex: LLaMA 3, GPT-4, Mistral).
- Treinado em trilhões de tokens (toda a internet).

2 Modelo Sintonizado com Instruções (Instruction-Tuned):

- O modelo de base é afinado (fine-tuned) num dataset de alta qualidade de "instrução-resposta" (ex: "Escreve um poema sobre...") e muitas vezes com feedback humano (RLHF).
- Ex: ChatGPT, LLaMA 3-Instruct. É isto que os torna "úteis" e "conversacionais".

3 Modelo de Tarefa (Task-Tuned com LoRA):

- Pegamos no modelo -Instruct e aplicamos LoRA para o especializar nos *nossos* dados (ex: o nosso Q&A de matemática, documentos financeiros da nossa empresa, etc.).

Resumo da Aula I

Resumo da Aula II

O que Vimos Hoje

- Revimos a nossa implementação de **Seq2Seq com Atenção** para Q&A.
- Apresentámos a arquitetura **GPT**, baseada apenas nos Decoders do Transformer (com atenção mascarada).
- Discutimos o objetivo de treino **Causal Language Model (CLM)**, focado em prever a próxima palavra, que permite a geração de texto.
- Vimos como o "In-Context Learning"(few-shot) é um comportamento emergente.
- Identificámos os problemas do "Full Fine-Tuning"(custo, armazenamento).
- Introduzimos o **PEFT** e o **LoRA** como a solução moderna e eficiente para adaptar LLMs a tarefas específicas, treinando

Referências I